

# HelixCode

## HelixCode

**La plataforma de desarrollo distribuido AI que divide el trabajo, lo preserva y nunca pierde tu progreso.**

### Resumen

HelixCode es una plataforma de desarrollo distribuido AI de nivel empresarial basada en Go, que divide el trabajo de desarrollo en tareas inteligentes distribuidas a través de una red de trabajadores gestionados por SSH, con puntos de control automáticos y retroceso para garantizar que ningún avance se pierda. Unifica la integración con múltiples proveedores de LLM, flujos de trabajo completos del ciclo de desarrollo y entrega multiplataforma bajo interfaces REST, CLI, TUI y MCP.

### Descripción breve

HelixCode es una plataforma de desarrollo distribuido AI escrita en Go. Divide el trabajo en tareas inteligentes a través de redes de trabajadores basados en SSH, preserva el progreso con puntos de control automáticos y retroceso, integra múltiples proveedores de LLM y gestiona el ciclo completo de desarrollo mediante interfaces REST, CLI, TUI y MCP.

### Descripción detallada

HelixCode es una plataforma de desarrollo distribuido AI de nivel empresarial (dev.helix.code, MIT) construida en torno a una promesa que su eslogan hace literal: dividir el trabajo, preservarlo y no perder nunca el hilo. Está diseñada para la división inteligente de tareas, la preservación automática del trabajo y los flujos de desarrollo multiplataforma, y está escrita en Go para satisfacer las demandas de concurrencia y portabilidad de un

único binario que exige la computación distribuida, con puntos de control automáticos, retroceso y monitoreo en tiempo real como primitivas de primera clase, no como complementos opcionales.

Su arquitectura superpone una capa de interfaz REST + WebSocket + MCP API sobre un conjunto de servicios básicos especializados: autenticación y gestión de sesiones JWT, gestión de grupos de trabajadores basados en SSH con monitoreo de salud, gestión de tareas con puntos de control y manejo de dependencias, gestión de proyectos y flujos de trabajo, y una capa unificada de proveedores LLM, todo ello persistente en PostgreSQL, con Redis disponible como nivel opcional de coordinación y almacenamiento en caché. Los trabajadores distribuidos se instalan automáticamente en una red, por lo que escalar el sistema es tan sencillo como indicar al servidor una máquina en lugar de configurarla manualmente. Las interfaces multiusuario abarcan CLI, interfaz de terminal, REST y frameworks móviles, de modo que la misma plataforma es accesible desde un script, una terminal o una aplicación.

HelixCode gestiona un ciclo de desarrollo completo de principio a fin: los flujos de planificación, construcción, pruebas y refactorización se ejecutan automáticamente con conciencia de dependencias y seguimiento de contexto multisecuencial, de manera que un esfuerzo de larga duración mantiene su continuidad incluso ante interrupciones o cambios de máquina. Integra múltiples proveedores de LLM —llama.cpp, Ollama y OpenAI— bajo una única interfaz, añade selección de modelos con conciencia de hardware que detecta CPU/GPU/memoria disponibles y adapta el modelo a la máquina, y soporta estrategias de razonamiento avanzadas como *chain-of-thought* y *tree-of-thoughts* para problemas que requieren más de un solo paso. El Model Context Protocol se implementa a través de múltiples transportes para el intercambio estandarizado de herramientas y contexto, y las notificaciones multicanal (Slack, Discord, correo electrónico, Telegram) mantienen informados a los equipos mientras avanza el trabajo distribuido. Es compatible con Linux, macOS, Windows, Aurora OS y SymphonyOS.

## Por qué lo creamos

El desarrollo distribuido y asistido por AI suele perder contexto y progreso cuando las tareas se dividen entre máquinas o se interrumpen. HelixCode se diseñó para hacer que la división de tareas sea inteligente y la preservación del trabajo, automática. Así, un esfuerzo de desarrollo a gran escala puede desglosarse, distribuirse en una red de trabajadores, establecer puntos de control y reanudarse o revertirse sin perder el estado.

# Por qué es un cambio radical

Convierte el desarrollo distribuido con AI en algo *duradero*: una capacidad que nunca fue práctica cuando los equipos unían estas piezas manualmente. Tres elementos que normalmente existen en tres herramientas distintas se integran en una sola plataforma: cómputo distribuido (redes de trabajadores SSH con instalación automática y monitoreo de salud), asistencia para desarrollo con AI (LLM multiproveedor con razonamiento y llamadas a herramientas) y automatización de flujos de trabajo de ciclo completo. El tejido conectivo es el establecimiento de puntos de control respaldado por base de datos: como el estado de las tareas, los puntos de control y las dependencias se almacenan en PostgreSQL, un trabajo que abarca múltiples máquinas y sesiones puede revertirse o reanudarse exactamente donde se detuvo. Las interrupciones y el trabajo dividido dejan de ser una fuente de pérdida de progreso para convertirse en un evento rutinario y recuperable.

## Qué hay de innovador

- La preservación del trabajo como primitiva central: establecimiento automático de puntos de control y reversión aplicados a tareas de desarrollo *distribuidas*, de modo que el progreso sobrevive a interrupciones y fallos de máquina en lugar de desaparecer con ellos.
- Selección de modelos con conciencia de hardware que inspecciona la CPU/GPU/memoria detectada y asigna cada tarea a un modelo que la máquina pueda ejecutar de manera óptima, sin necesidad de ajustes manuales por trabajador.
- Una plataforma, cinco puertas de entrada: REST, WebSocket, CLI, TUI y MCP, con MCP expuesta a través de múltiples protocolos para que herramientas y agentes puedan integrarse según su forma de conexión.
- Alcance multiplataforma que va más allá del trío de escritorio habitual para incluir Aurora OS y SymphonyOS, ampliando la flota de trabajadores a plataformas que la mayoría de las herramientas ignoran.

## Mayores desafíos técnicos y cómo los resolvimos

- **No perder trabajo en tareas distribuidas e interrumpibles.** Cuando un trabajo se divide entre máquinas, cualquier fallo o interrupción suele dejar varado lo que estaba en ejecución. Modelamos la tarea misma como portadora de puntos de control y dependencias, almacenados en PostgreSQL, de modo que el sistema puede revertirse al último

estado válido o reanudarse desde él: una durabilidad que reside en la capa de datos, no en un frágil estado en memoria.

- **Gestionar una flota de trabajadores heterogénea.** Una red de máquinas con Linux, macOS, Windows, Aurora y SymphonyOS es un objetivo en constante cambio en cuanto a disponibilidad y configuración. Lo manejamos con un servicio dedicado de agrupación de trabajadores que realiza registro basado en SSH, instalación automática en nuevos nodos y monitoreo continuo de salud, manteniendo así la flota conocida y controlable a medida que las máquinas entran y salen.
- **Heterogeneidad de proveedores y hardware.** Los backends de LLM y las máquinas que los ejecutan varían enormemente en capacidades. Lo ocultamos tras una interfaz unificada de proveedores de LLM y lo combinamos con detección de hardware (CPU/GPU/memoria) que impulsa una selección inteligente de modelos, de modo que el modelo adecuado se despliega en la máquina correcta sin que quien lo invoca deba razonar sobre ninguno de los dos aspectos.

## Pila tecnológica

- **Go (1.26+ módulo interno)** — seleccionado por su concurrencia basada en goroutines y su salida en un único binario, exactamente lo que necesita un sistema de trabajadores distribuidos: paralelismo económico para la orquestación y un binario autónomo que se autoinstala en cualquier nodo. Incluye todos los servicios principales y los binarios CLI/server.
- **Gin (marco HTTP)** — elegido por ofrecer una capa REST rápida y minimalista, con baja sobrecarga; sirve la superficie / api/v1 (autenticación, trabajadores, tareas, proyectos) con la que interactúa cada cliente.
- **PostgreSQL 15+ (vía pgx/v5)** — seleccionado como sistema de registro duradero, ya que el checkpointing y el rollback exigen persistencia transaccional; alberga el esquema de computación distribuida de 11 tablas (usuarios, trabajadores, tareas, proyectos, sesiones, llm\_providers, notificaciones) que hace posible la preservación del trabajo.
- **Redis 7+ (opcional, go-redis/v9)** — elegido como capa opcional de caché y coordinación que acelera los caminos críticos sin convertirse en una dependencia obligatoria, de modo que una implementación mínima funcione únicamente con Postgres.
- **SSH** — seleccionado como transporte de control de trabajadores precisamente por estar ya ampliamente disponible y ser seguro; gestiona el registro de trabajadores, la autoinstalación y la ejecución remota de comandos en todo el pool sin necesidad de desplegar primero un agente personalizado.

- **Model Context Protocol (MCP)** — elegido para el intercambio estandarizado de herramientas y contextos, permitiendo que herramientas y agentes externos se integren mediante un protocolo abierto; implementado con soporte multitransporte para adaptarse a los clientes dondequiera que se conecten.
- **Proveedores LLM (llama.cpp, Ollama, OpenAI)** — seleccionados para abarcar tanto la inferencia local como la alojada bajo una única interfaz unificada, de modo que la selección consciente del hardware pueda dirigir una tarea a un modelo local o alojado sin que el solicitante note la diferencia.

## Estado y notas de transparencia

- **Estado: beta.** El archivo README se autodefine como “COMPLETAMENTE FINALIZADO / todas las 5 fases”; sin embargo, esta completitud es declarada por el proyecto y no ha sido verificada de forma independiente, por lo que se considera en fase beta.
- Todos los detalles anteriores provienen del archivo README del repositorio; las frases de marketing (eslóganes) son de carácter editorial y no reflejan métricas técnicas de origen.

**Nivel de prioridad:** Helix principal.